

LeetCode 914: X of a Kind in a Deck of Cards

Algorithm Analysis & Implementation

Problem Overview

The core objective of this problem is to determine whether it is possible to partition a given deck of cards into groups such that every group consists of exactly X cards (where $X \geq 2$). Furthermore, all cards within any single group must carry the exact same integer value. To achieve this state, the total frequency count of every individual card number must be cleanly divisible by the universal group size X .

Theoretical Approach

This problem can be elegantly solved by fusing frequency mapping with the mathematical concept of the **Greatest Common Divisor (GCD)**. The methodology involves two fundamental phases:

1. Frequency Hashing

Initially, we traverse the entire deck to tally the occurrences of each card. A Hash Map (`unordered_map` in C++) is the optimal data structure here. The key represents the integer value on the card, while the value maintains its overall frequency in the deck.

2. Finding the GCD

For a universal group size X to exist, it must be a common divisor for all the calculated frequencies. To maximize our chances (and to encompass all valid smaller group sizes), we look for the Greatest Common Divisor (GCD) of all these frequencies. We iterate through the frequency map, maintaining a running GCD across all count values.

Conclusion Criteria: *If the final computed GCD is at least 2, it guarantees that a valid partitioning strategy exists, and we yield `true`. Conversely, if the*

GCD resolves to 1, no such common grouping size is mathematically possible, yielding false.

Complexity Analysis

- **Time Complexity: $O(N \cdot \log K)$**

Here, N represents the total number of cards, and K is the maximum frequency count. Building the hash map operates in $O(N)$ time. Sequentially calculating the GCD across unique elements utilizes the Euclidean algorithm, completing in logarithmic time relative to the frequencies.

- **Space Complexity: $O(N)$**

In the worst-case scenario where all elements are uniquely distinct, the hash map scales linearly with the input deck size to store every individual mapping.

Source Code

Below is the preserved algorithm, formatted for standard readability and indentation:

```
class Solution {
public:
    bool hasGroupsSizeX(vector<int>& deck) {
        unordered_map<int, int> mp;
        int n = deck.size();

        if (n <= 1) return false;

        // Map card values to their frequencies
        for (int i = 0; i < n; i++) {
            mp[deck[i]]++;
        }

        int g = 0;
        // Calculate the GCD of all frequencies
        for (auto it = mp.begin(); it != mp.end(); it++) {
            g = __gcd(g, it->second);
        }

        return g >= 2;
    }
};
```

```
}  
};
```

© Authored by Arpit Raj, LNMIIT Jaipur | July 7, 2026